

ΔΟΜΕΣ ΔΕΔΟΜΕΝΩΝ ΕΡΓΑΣΙΑ 2(p3220273,p3210271)

Για το μέρος Α της εργασίας ,χρησιμοποιήσαμε την Heapsort η οποία είναι μια ταξινόμηση η οποία ανήκει στην κατηγορία των δέντρων και βασίζεται στην φιλοσοφία του δυαδικού δέντρου (binary heap) και λειτουργεί ως εξής:

Στον παραπάνω κώδικα η μέθοδος heapsort καλείτε από το πρόγραμμα με μεταβλητές εισόδου την λίστα cities που περιέχει αντικείμενα τύπου City και με την σειρά του η heapsort καλεί την heapify με μεταβλητές εισόδου την προαναφέρθουσα λίστα cities, το int s που είναι το μέγεθος του πίνακα και την μεταβλητή i που είναι δείκτης του heap μας. Έπειτα αρχικοποιούμε το heap μας (γονέα) και το αριστερό και δεξί δείκτη(παιδιά του heap) και μέσω των πρώτων 2 if statements προσπαθούμε να ταξινομήσουμε τον πίνακα έτσι ώστε να βρούμε το μέγιστο heap (max heap) οπού αυτός είναι ο μέγιστη τιμή ανάμεσα στους κόμβους του “δέντρου”. Έπειτα γίνεται η αφαίρεση του max heap από το δέντρο. Αυτό επιτυγχάνεται μέσω του 3ου if statement που τοποθετεί το max heap στο τέλος της ουράς και έπειτα συρρικνώνει τον πίνακα κατά ένα και έτσι “διώχνουμε” το πρώην max heap του από το “δέντρο”. Στην συνέχεια η heapify ανακαλεί τον εαυτό της(με μεταβλητές εισόδου την λίστα cities, το μέγεθος του πίνακα s και το νέο heap) έτσι ώστε να βρει το νέο max heap της με τα υπόλοιπα στοιχεία που έχουν μείνει στον πίνακα. Αυτή η διαδικασία επαναλαμβάνεται μέχρι να ταξινομηθούν όλα τα στοιχεία. Στο τέλος της ταξινόμησης, ο πίνακας είναι ταξινομημένος σε αύξουσα σειρά.

Για το μέρος Β, η μέθοδος remove τύπου City δέχεται ως παράμετρο την μεταβλητή τύπου int “ID” η οποία είναι η ID των πόλεων. Με την χρήση της βοηθητικής μεταβλητής hsize (που αρχικοποιούμε με το 0) ενώ εκτελείτε σάρωση στον πίνακα PQueue θα βρούμε (αν υπάρχει)την θέση του πρέπει να αφαιρεθεί αποθηκεύοντας σε αυτήν την θέση του. Σε περίπτωση που δεν βρέθηκε, (δηλαδή το hsize παραμένει 0) επιστρέφει null, σε διαφορετική περίπτωση γίνεται αντικατάσταση του στοιχείου στην θέση που είναι αποθηκευμένη στην hsize και της size (μέγεθος της ουράς) με την βοήθεια της βοηθητικής μεταβλητής CityR και έπειτα μικραίνει το πίνακα κατά 1 αφαιρώντας έτσι το στοιχείου που ήταν πριν στο σημείο της hsize(η μεταβλητή όμως αυτή είναι ακόμα αποθηκευμένη στην μεταβλητή CityR).Έπειτα καλείτε η μέθοδος heapify που δέχεται της θέση hsize για να επαναφέρει την ιδιότητα του max heap. Έπειτα η μέθοδος επιστρέφει την μεταβλητή CityR(του City). Εν συντομία, η remove αφαιρεί ένα στοιχείο βάση του ID που την δίνεται, μειώνοντας το μέγεθος του πίνακα και ανακατασκευάζει τον buffer για να επαναφέρει την ιδιότητα του max heap.

Για το Γ γενικά, το κυρίως μέρος του προγράμματος βρίσκεται στην main μέθοδο, όπου γίνεται η ανάγνωση του αρχείου εισόδου γραμμή προς γραμμή.

Κάθε φορά που διαβάζονται 5 γραμμές, οι κορυφαίες k πόλεις (βάσει του τελεστή σύγκρισης) εκτυπώνονται.

Η πολυπλοκότητα αυτού του προγράμματος εξαρτάται από τον αριθμό των πόλεων συνολικά (n) και τον αριθμό k . Η ανάγνωση του αρχείου είναι γραμμική ως προς τον αριθμό των γραμμών, άρα $O(n)$.

Κάθε φορά που εκτυπώνονται οι κορυφαίες k πόλεις, εκτελείται μια λειτουργία εισαγωγής στον PriorityQueue, που έχει πολυπλοκότητα $O(\log k)$.

Συνεπώς, η συνολική πολυπλοκότητα του προγράμματος είναι $O(n \log k)$. Αν το k είναι αρκετά μικρό σε σχέση με τον συνολικό αριθμό των πόλεων, το κόστος της εισαγωγής στην ουρά προτεραιότητας θα είναι χαμηλό και η συνολική απόδοση του προγράμματος θα είναι αποδεκτή.

Αν όμως το k είναι κοντά στον συνολικό αριθμό των πόλεων, το κόστος της εισαγωγής στην ουρά προτεραιότητας θα είναι υψηλό και θα χρειαστεί περισσότερος χρόνος εκτέλεσης.

Σε γενικές γραμμές, αν το k είναι πολύ μικρό σε σχέση με τον συνολικό αριθμό των πόλεων (n), τότε η προσέγγιση με του DynamicInfluenza_k_withPQ πιθανότατα θα είναι πιο αποδοτική, καθώς δεν απαιτεί την πλήρη ταξινόμηση όλων των πόλεων.

Αντίθετα, αν το k είναι κοντά στο n , τότε η προσέγγιση της Influenza_k μπορεί να είναι πιο αποδοτική, καθώς η ταξινόμηση γίνεται μόνο μία φορά.

Μερos Γ αναλυτικά:

πρωτα ελέγχει αν οι παράμετροι εισόδου είναι σωστοί. Αν δεν υπάρχουν δύο παράμετροι (k , αρχείο εισόδου), τότε εκτυπώνεται ένα μήνυμα λάθους και το πρόγραμμα τερματίζεται.

Επειτα κάνει Ανάγνωση Δεδομένων από το Αρχείο οπου Δημιουργεί ένα αντικείμενο PriorityQueue<City> για να διαχειριστεί τις πόλεις βάσει της προτεραιότητάς τους.

Μετα Διαβάζει τις γραμμές από το αρχείο εισόδου. Κάθε γραμμή περιέχει πληροφορίες για μια πόλη και

Δημιουργεί ένα αντικείμενο City για κάθε πόλη και το προσθέτει στο PriorityQueue.

Υστερα Κατά τη διάβαση κάθε 5ης γραμμής, υπολογίζει την πυκνότητα για κάθε πόλη με βάση τα κρούσματα γρίπης και τον πληθυσμό.

Προσθέτει την πόλη στο PriorityQueue και εκτυπώνει τις κορυφαίες k πόλεις από το PriorityQueue.

Η κλάση City αποθηκεύει πληροφορίες για μια πόλη (ID, όνομα, πληθυσμός, κρούσματα γρίπης) και

υλοποιεί τον τελεστή σύγκρισης (compareTo) για να συγκρίνει τις πόλεις βάσει της πυκνότητας, του ονόματος και του αναγνωριστικού τους.